

Insurance Claims Support AI Agent with LangMem and RAG

GitHub Repository:

https://github.com/yashprogrammer/cutomer_support_agent_langmem.git

2. Project Duration

Approximate total effort: **6–8 hours**

- **Setup and environment configuration:** 45–60 minutes
 - **LangMem memory integration and tool-calling flow:** 90–120 minutes
 - **Modular FastAPI backend and dashboard implementation:** 150–180 minutes
 - **Testing, Dockerization, CI/CD, and EC2 deployment setup:** 90–120 minutes
-

3. Introduction

This project implements an **internal AI copilot** that assists human support agents and insurance claims adjusters in handling **First Notice of Loss (FNOL)** and related claim-support workflows.

The copilot retrieves relevant claim history, policy/knowledge-base content, and structured tool outputs, then generates an **editable draft recommendation** for a human reviewer.

Final decisions remain with **licensed adjusters or support agents**.

When a recommendation is **accepted**, the approved resolution is written back into the **memory layer** so future claim handling becomes more context-aware and consistent.

4. Aim

The goal of this project is to design and implement a **production-style AI claims-support pipeline** that can:

- Ingest and index insurance knowledge documents
 - Retrieve relevant customer and company claim history from memory
 - Search policy and claims guidance using **RAG**
 - Call structured operational tools during draft generation
 - Generate concise, grounded, editable AI recommendations
 - Keep a **human-in-the-loop** for review and approval
 - Persist accepted resolutions back into memory for reuse in future cases
-

5. Dataset and Data Sources

This project uses a **hybrid strategy** combining static domain knowledge with runtime operational data.

5.1 Knowledge Base Files

Domain knowledge is stored under **knowledge_base/** as Markdown files.

Current examples include:

- `insurance-auto-claims-fnol-intake-checklist.md`
- `insurance-auto-coverage-and-deductible-guidelines.md`
- `insurance-auto-required-documents-by-claim-type.md`
- `insurance-claims-settlement-sla-and-communication.md`
- `insurance-claims-fraud-risk-indicators.md`

These files are **chunked and indexed into ChromaDB** to support retrieval-augmented generation.

5.3 Memory Data

Long-term working memory is maintained through **LangMem on top of LangGraph InMemoryStore**.

Memory is organized into:

- **Customer-level memory scope** using normalized email
- **Company-level memory scope** using `company::<slug>`

Accepted recommendations are written into these scopes as reusable claim-resolution memories.

Important implementation detail:

- If **GOOGLE_API_KEY** is configured, **semantic memory search** is enabled using Gemini embeddings.
 - If it is not configured, the memory layer still works but falls back to **non-semantic recent-memory listing behavior**.
-

6. Tools and Technologies

Core Backend

- Python 3.11
- FastAPI
- Uvicorn
- Pydantic and **pydantic-settings**

AI and Orchestration

- LangChain **create_agent**
- LangGraph **InMemorySaver**
- Groq via **langchain-groq**
- LangMem for memory management
- LangGraph **InMemoryStore**

Retrieval and Embeddings

- ChromaDB
- **langchain-text-splitters**
- Gemini embeddings via **langchain-google-genai** and **google-genai**

Data and UI

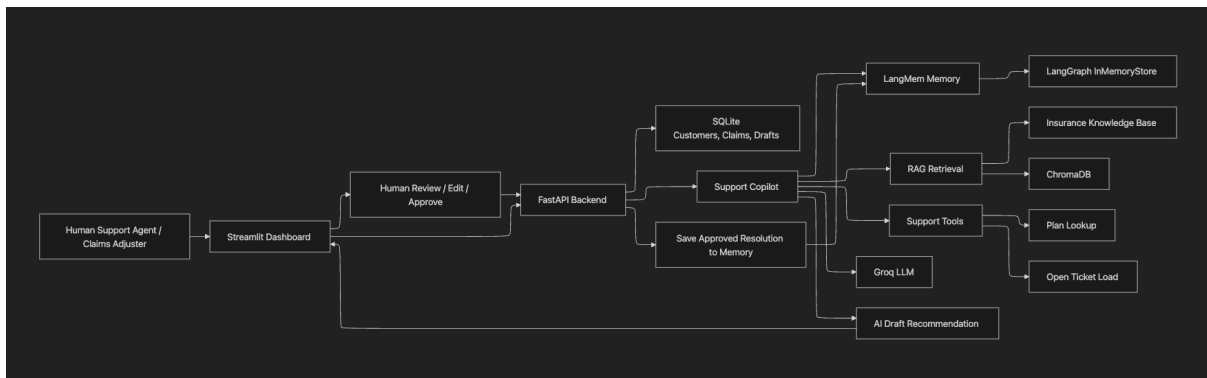
- SQLite
- Streamlit

DevOps and Tooling

- **uv**
- Docker
- Docker Compose

- Pytest
- GitHub Actions
- AWS EC2

7. System Architecture



8. Key Modules and Implementation Details

8.1 Project Bootstrap

- Environment management through `.env`
- Centralized configuration in `customer_support_agent/core/settings.py`
- Automatic directory creation for database and vector-store paths
- FastAPI application bootstrap in `customer_support_agent/api/app_factory.py`

8.2 Data Model Layer

SQLite tables are initialized for:

- `customers`
- `tickets`
- `drafts`

The repository layer provides modular access to customer, ticket, and draft operations.

8.3 Knowledge Ingestion and RAG

The knowledge layer is implemented in:

`customer_support_agent/integrations/rag/chroma_kb.py`

It performs:

- file loading from `knowledge_base/`
- chunking using `RecursiveCharacterTextSplitter`
- embedding generation
- Chroma collection upsert
- top-k semantic retrieval with source metadata

If a **Google API key** is available, the RAG layer uses **Gemini embeddings**. Otherwise, it falls back to **Chroma's default embedding function**.

8.4 LangMem-Based Memory Layer

The memory layer is implemented in:

`customer_support_agent/integrations/memory/langmem_store.py`

Key design points:

- Uses LangMem `create_manage_memory_tool` for memory creation
- Stores data in LangGraph `InMemoryStore`
- Supports customer and company scope retrieval
- Normalizes results into `memory`, `score`, and `metadata`
- Deduplicates repeated hits across scopes
- Falls back to recent-memory listing if semantic search returns no match
- Preserves operation continuity even when semantic indexing is unavailable

8.5 Tool Calling Layer

Structured tools are defined in:

`customer_support_agent/integrations/tools/support_tools.py`

Current tools include:

- `lookup_customer_plan`
- `lookup_open_ticket_load`

These tools help the agent enrich recommendations with **operational facts** such as plan tier, SLA expectations, and current open-ticket volume.

8.6 Copilot Orchestration

The main orchestration logic lives in:

`customer_support_agent/services/copilot_service.py`

The draft generation pipeline includes:

- memory retrieval
- knowledge retrieval
- tool invocation through the agent runtime
- system prompt composition
- user prompt composition
- Groq-based draft generation
- fallback generation logic if the primary response is empty
- structured context capture for transparency

8.7 Draft Lifecycle

The draft lifecycle is managed through routers and **DraftService**.

Capabilities include:

- background draft generation at ticket creation
- manual draft generation on demand
- editable draft storage
- acceptance and discard workflow
- structured context tracking in `context_used`

8.8 Human-in-the-Loop Workflow

The system is intentionally designed so the **AI does not make autonomous final claim decisions**.

The human reviewer:

- reads the generated recommendation
- edits it if needed
- approves or discards it
- controls whether the accepted recommendation becomes future memory

This is important in insurance workflows where **claim decisions require compliance, judgment, and accountability**.

8.9 Streamlit Dashboard

The dashboard in `app.py` provides:

- claim/FNOL registration
- claim listing and selection
- recommendation generation
- draft editing
- approval and request-for-info actions
- knowledge ingestion button
- claim-history memory probing
- context visualization for memory hits, knowledge hits, tool calls, and errors

9. Learning Outcomes

After completing this project, a learner can demonstrate the ability to:

- design a modular AI application using routers, services, repositories, and integrations
- build REST APIs using FastAPI
- implement an insurance-support RAG workflow using ChromaDB
- integrate LangMem-based long-term memory into an AI application
- use Groq as an LLM backend through LangChain
- combine memory, RAG, and tool calling in one orchestration pipeline
- build a Streamlit dashboard for operational AI workflows
- implement human-in-the-loop approval for sensitive AI outputs
- containerize and deploy a multi-component AI application
- add automated tests and CI/CD support for production-style delivery

16. Resource link

Learn RAG basics ->

 **Retrieval Augmented Generation | What is RAG | How does RAG Work | RAG Ex...**

RAG Evaluations->  **RAG Evaluation Using DeepEval & Confident AI — Full Tutorial**

langMem Docs -> <https://langchain-ai.github.io/langmem/>